

Question 1. Sequence Comparison For this question, you will work with a one megabase segment of human chromosome 22.

Download, unzip it and extract one megabase segment from 20Mb to 21Mbp:

Q1a. Write a python script (sim_mutations.py) to read in a fasta file, introduce a specified level of mutations, and then output a new fasta file. The input (-i) and output (-o) filenames should be set on the command line as should the level of mutations (-m) and the random seed (-s). For mutations, only introduce substitutions by changing the input base to a different base at random. The substitutions should be inserted at random locations at the specified rate. For example "-m 0.015" should change 1.5% of the original bases.

```
#Sim_mutations.py

import argparse
import random as rng

#Need parser to allow for CLI flags
parser = argparse.ArgumentParser(description="sim_mutations argument parser")

parser.add_argument('-i', '--input', type=str, help="path to input file")
parser.add_argument('-o', '--output', type=str, help="path to output file")
parser.add_argument('-m', '--mutation', type=float, help="proportion of input bases to change to different base")
parser.add_argument('-s', '--seed', type=int, help="random seed", default=42)

args = parser.parse_args()

#Open required files
input_file = open(args.input)
output_file = open(args.output, 'w')

#Copy over the fasta header line
output_file.write(f"{input_file.readline().strip()}_mutated\n")

sequence = ''.join(line.strip().upper() for line in input_file.readlines())
length = len(sequence)

sequence = list(sequence)
edits = int(args.mutation * length)

rng.seed(args.seed)
bases_to_edit = rng.sample(range(length), edits)

bases = ['A', 'C', 'G', 'T']
#Replacing each base with some mutated one
for index in bases_to_edit:
```

```

original = sequence[index]

# Don't mutate N
if original == 'N':
    continue

# Choose a base different from the original

possible_bases = [base for base in bases if base != original]
sequence[index] = rng.choice(possible_bases)

# Convert back to a string
sequence = ''.join(sequence)

#Write lines 50 characters per line
line_length = 50
output_file.writelines(f"{sequence[p:p + line_length]}\n" for p in range(0, length, line_length))

#Close files at end
input_file.close()
output_file.close()

```

Q1b. Now run the script using error rates 0.01, 0.05, 0.10 and 0.15 using three random seeds(e.g. 1, 2, 3), and compare the simulated mutation rate to the average identity (%IDY) computed by mummer. You may find this bash code helpful to evaluate all the runs. Note, with a mutation rate of 0.15 we expect the average identity to be 85%. Some deviation is expected because of rounding, plus mummer may choose to not extend an alignment to the very end of a sequence if there is a cluster of mutations

make_mutations.sh:

```

for rate in 0.01 0.05 0.10 0.15
do
    for seed in 1 2 3
    do
        if [ ! -r chr22_mut${rate}_s${seed}.fa ]
        then
            # introduce mutations
            python3 sim_mutations.py -i chr22_orig.fa -o chr22_mut${rate}_s${seed}.fa -m ${rate} -s ${seed}
            nucmer --mum -g 1000 -b 1000 -p chr22_orig_vs_mut${rate}_s${seed} chr22_orig.fa chr22_mut${rate}_s${seed}.fa
            delta-filter -1 chr22_orig_vs_mut${rate}_s${seed}.delta > chr22_orig_vs_mut${rate}_s${seed}.1delta
            show-coords -rcl chr22_orig_vs_mut${rate}_s${seed}.1delta > chr22_orig_vs_mut${rate}_s${seed}.coords
        fi
    done
done

```

```
head chr22_orig_vs_mut*.coords
```

Answers:

```

“ ==> chr22_orig_vs_mut0.01_s1.coords <== /Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/chr22_orig.fa /Users/ashrithkollu/Docum
ACG/Assignment2/chr22_mut0.01_s1.fa NUCMER

```

[S1] [E1] | [S2] [E2] | [LEN 1] [LEN 2] | [% IDY] | [LEN R] [LEN Q] | [COV R] [COV Q] | [TAGS]

```

1 1000001 | 1 1000001 | 1000001 1000001 | 99.00 | 1000001 1000001 | 100.00 100.00 | chr22:20000000-21000000 c
==> chr22_orig_vs_mut0.01_s2.coords <== /Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/chr22_orig.fa /Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/chr22_mut0.01_s2.fa NUCMER

```

[S1] [E1] | [S2] [E2] | [LEN 1] [LEN 2] | [% IDY] | [LEN R] [LEN Q] | [COV R] [COV Q] | [TAGS]

```

1 1000001 | 1 1000001 | 1000001 1000001 | 99.00 | 1000001 1000001 | 100.00 100.00 | chr22:20000000-21000000 c
==> chr22_orig_vs_mut0.01_s3.coords <== /Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/chr22_orig.fa /Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/chr22_mut0.01_s3.fa NUCMER

```

[S1] [E1] | [S2] [E2] | [LEN 1] [LEN 2] | [% IDY] | [LEN R] [LEN Q] | [COV R] [COV Q] | [TAGS]

```

1 1000001 | 1 1000001 | 1000001 1000001 | 99.00 | 1000001 1000001 | 100.00 100.00 | chr22:20000000-21000000 c
==> chr22_orig_vs_mut0.05_s1.coords <== /Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/chr22_orig.fa /Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/chr22_mut0.05_s1.fa NUCMER

```

[S1] [E1] | [S2] [E2] | [LEN 1] [LEN 2] | [% IDY] | [LEN R] [LEN Q] | [COV R] [COV Q] | [TAGS]

```

1 1000001 | 1 1000001 | 1000001 1000001 | 95.00 | 1000001 1000001 | 100.00 100.00 | chr22:20000000-21000000 c
==> chr22_orig_vs_mut0.05_s2.coords <== /Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/chr22_orig.fa /Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/chr22_mut0.05_s2.fa NUCMER

```

[S1] [E1] | [S2] [E2] | [LEN 1] [LEN 2] | [% IDY] | [LEN R] [LEN Q] | [COV R] [COV Q] | [TAGS]

1	1000001		1	1000001		1000001	1000001		95.00		1000001	1000001		100.00	100.00		chr22:20000000-21000000	c
---	---------	--	---	---------	--	---------	---------	--	-------	--	---------	---------	--	--------	--------	--	-------------------------	---

==> chr22_orig_vs_mut0.05_s3.coords <== /Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/chr22_orig.fa /Users/ashrithkollu/Documents/ACG/Assignment2/chr22_mut0.05_s3.fa NUCMER

[S1] [E1] | [S2] [E2] | [LEN 1] [LEN 2] | [% IDY] | [LEN R] [LEN Q] | [COV R] [COV Q] | [TAGS]

1 1000001 | 1 1000001 | 1000001 1000001 | 95.00 | 1000001 1000001 | 100.00 100.00 | chr22:20000000-21000000 c

==> chr22_orig_vs_mut0.10_s1.coords <== /Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/chr22_orig.fa /Users/ashrithkollu/Documents/ACG/Assignment2/chr22_mut0.10_s1.fa NUCMER

[S1] [E1] | [S2] [E2] | [LEN 1] [LEN 2] | [% IDY] | [LEN R] [LEN Q] | [COV R] [COV Q] | [TAGS]

1 1000001 | 1 1000001 | 1000001 1000001 | 90.02 | 1000001 1000001 | 100.00 100.00 | chr22:20000000-21000000 c

==> chr22_orig_vs_mut0.10_s2.coords <== /Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/chr22_orig.fa /Users/ashrithkollu/Documents/ACG/Assignment2/chr22_mut0.10_s2.fa NUCMER

[S1] [E1] | [S2] [E2] | [LEN 1] [LEN 2] | [% IDY] | [LEN R] [LEN Q] | [COV R] [COV Q] | [TAGS]

1 999996 | 1 999996 | 999996 999996 | 90.02 | 1000001 1000001 | 100.00 100.00 | chr22:20000000-21000000 c

==> chr22_orig_vs_mut0.10_s3.coords <== /Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/chr22_orig.fa /Users/ashrithkollu/Documents/ACG/Assignment2/chr22_mut0.10_s3.fa NUCMER

[S1] [E1] | [S2] [E2] | [LEN 1] [LEN 2] | [% IDY] | [LEN R] [LEN Q] | [COV R] [COV Q] | [TAGS]

1 999973 | 1 999973 | 999973 999973 | 90.02 | 1000001 1000001 | 100.00 100.00 | chr22:20000000-21000000 c

==> chr22_orig_vs_mut0.15_s1.coords <== /Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/chr22_orig.fa /Users/ashrithkollu/Documents/ACG/Assignment2/chr22_mut0.15_s1.fa NUCMER

[S1] [E1] | [S2] [E2] | [LEN 1] [LEN 2] | [% IDY] | [LEN R] [LEN Q] | [COV R] [COV Q] | [TAGS]

1 1000001 | 1 1000001 | 1000001 1000001 | 85.09 | 1000001 1000001 | 100.00 100.00 | chr22:20000000-21000000 c

==> chr22_orig_vs_mut0.15_s2.coords <== /Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/chr22_orig.fa /Users/ashrithkollu/Documents/ACG/Assignment2/chr22_mut0.15_s2.fa NUCMER

[S1] [E1] | [S2] [E2] | [LEN 1] [LEN 2] | [% IDY] | [LEN R] [LEN Q] | [COV R] [COV Q] | [TAGS]

1 999996 | 1 999996 | 999996 999996 | 85.09 | 1000001 1000001 | 100.00 100.00 | chr22:20000000-21000000 c

```
==> chr22_orig_vs_mut0.15_s3.coords <== /Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/chr22_orig.fa /Users/ashrithkollu/Documents/ACG/Assignment2/chr22_mut0.15_s3.fa NUCMER
```

[S1]	[E1]	[S2]	[E2]	[LEN 1]	[LEN 2]	[% IDY]	[LEN R]	[LEN Q]	[COV R]	[COV Q]	[TAGS]
1	999973	1	999973	999973	999973	85.09	1000001	1000001	100.00	100.00	chr22:20000000-21000000

The simulated mutation rate is nearly equal to the average identity computed by mummer. The average for 1% is 99% identity, the average for 5% is 95% identity, the average for 10% is 90.02% identity, the average for 15% is 85.09% identity. The mummer identity aligns nearly perfectly with what we expected from the simulations.

Q1c. Now write a python program to compute the Jaccard coefficient between two sequences specified with -a and -b using the specified kmer length (-k) Output the Jaccard coefficient, the ANI (computed by computing the kth root), and the approximate ANI computed using natural log. Convert any lowercase letters to uppercase, and convert any non-ACGT characters to N.

```
#compute_jaccard.py
```

```
def build_kmer_set(input_file, k_size):

    fastastart = input_file.readline() #throw out first line
    if (not fastastart.startswith(">")):
        input_file.seek(0)

    kmer_set = set()
    prev_k = "" #maintain k-1 previous characters to build kmer

    for line in input_file:
        line = list(line.strip().upper())

        for m in range(len(line)):
            if (line[m] not in ['A', 'C', 'G', 'T']):
                line[m] = 'N'

        line = ''.join(line)

        sequence = prev_k + line

        #Build k-mers and add to dictionary
        for i in range(len(sequence) - k_size + 1):
            kmer = sequence[i:i+k_size]
            kmer_set.add(kmer)
```

```

        #Overlap
        prev_k = sequence[-(k_size-1):]

    return kmer_set

import argparse
import math

#Need parser to allow for CLI flags
parser = argparse.ArgumentParser(description="Jaccard.py argument parser")

parser.add_argument('-a', '--sequence1', type=str, help="Sequence 1 to compare")
parser.add_argument('-b', '--sequence2', type=str, help="Sequence 2 to compare")
parser.add_argument('-k', '--kmer_length', type=int, help="length of kmer to compute Jaccard coefficient")

args = parser.parse_args()

k = args.kmer_length

#Open required files
input_file1 = open(args.sequence1)
input_file2 = open(args.sequence2)

#Build kmer index for sequence 1 and sequence 2
kmer_set_1 = build_kmer_set(input_file1, k)
kmer_set_2 = build_kmer_set(input_file2, k)

#Calculate and display outputs
jaccard = len(kmer_set_1.intersection(kmer_set_2)) / len(kmer_set_1.union(kmer_set_2))
ani = ((2*jaccard) / (1 + jaccard)) ** (1/k)
ani_approx = 1 + (1/k)*math.log((2*jaccard)/(1 + jaccard))
print(f"The Jaccard coefficient is {jaccard:.4f}.")
print(f"The Average Nucleotide Identity is exactly {ani:.4f}.")
print(f"The Average Nucleotide Identity is approximately {ani_approx:.4f}.")

input_file1.close()
input_file2.close()

```

Q1d. Run your Jaccard coefficient program on all the simulated sequences. Output a table of filename, expected identity, Jaccard, exact ANI, and approximate ANI. Comment on how well the ANI and approximate ANI compares to the expected level of mutation.

```
import os
os.chdir('/Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2')

#Code to create a table was made by ChatGPT given a prompt telling it the file name, structure, and desired info in the table.

import glob
import re

results = []

files = glob.glob("jaccard_mut*.txt")

for filename in files:
    # Get just the filename, without .txt
    name = filename[:-4]

    # Example: jaccard_mut0.01_s1
    match = re.fullmatch(r"jaccard_mut([0-9]+\.[0-9]+)_(s[0-9]+)", name)

    if match is None:
        print("Skipping:", filename)
        continue

    mutation_proportion = float(match.group(1))

    # Read file
    with open(filename, "r") as f:
        lines = f.readlines()

    # Extract values from the three lines
    jaccard = float(lines[0].split()[-1].rstrip("."))
    ani_exact = float(lines[1].split()[-1].rstrip("."))
    ani_approx = float(lines[2].split()[-1].rstrip("."))

    results.append([
        filename,
        mutation_proportion,
        jaccard,
        ani_exact,
        ani_approx
```

```

])

# Sort by mutation proportion, then filename
results.sort(key=lambda x: (x[1], x[0]))

# Display a Markdown table
print("| Filename | Expected Mutation Proportion | Jaccard Coefficient | ANI Exact | ANI Approximate |")
print("|---|---:|---:|---:|---:|")

for filename, mutation_proportion, jaccard, ani_exact, ani_approx in results:
    print(
        f"| {filename} | "
        f"{mutation_proportion:.4f} | "
        f"{jaccard:.6f} | "
        f"{ani_exact:.6f} | "
        f"{ani_approx:.6f} | "
    )

print(f"\nProcessed {len(results)} files.")

| Filename | Expected Mutation Proportion | Jaccard Coefficient | ANI Exact | ANI Approximate |
|---|---:|---:|---:|---:|
| jaccard_mut0.01_s1.txt | 0.0100 | 0.677600 | 0.989900 | 0.989800 |
| jaccard_mut0.01_s2.txt | 0.0100 | 0.676900 | 0.989900 | 0.989800 |
| jaccard_mut0.01_s3.txt | 0.0100 | 0.677400 | 0.989900 | 0.989800 |
| jaccard_mut0.05_s1.txt | 0.0500 | 0.208000 | 0.950500 | 0.949200 |
| jaccard_mut0.05_s2.txt | 0.0500 | 0.206400 | 0.950200 | 0.948900 |
| jaccard_mut0.05_s3.txt | 0.0500 | 0.206200 | 0.950200 | 0.948900 |
| jaccard_mut0.10_s1.txt | 0.1000 | 0.061670 | 0.902600 | 0.897500 |
| jaccard_mut0.10_s2.txt | 0.1000 | 0.060300 | 0.901700 | 0.896500 |
| jaccard_mut0.10_s3.txt | 0.1000 | 0.060580 | 0.901800 | 0.896700 |
| jaccard_mut0.15_s1.txt | 0.1500 | 0.018420 | 0.853800 | 0.841900 |
| jaccard_mut0.15_s2.txt | 0.1500 | 0.018210 | 0.853300 | 0.841400 |
| jaccard_mut0.15_s3.txt | 0.1500 | 0.018340 | 0.853600 | 0.841700 |

```

Processed 12 files.

Answer: Both the ANI and approximate ANI aligned with the expected level of mutation. At most, the approximate ANI was off by 0.0086 while the exact ANI was off by 0.0038. The difference was more significant in higher levels of mutation. These are miniscule differences and approximate the level of mutation quite well.

Q1e. Recall that modimizers are the subset of kmers such that $\text{hash}(\text{kmer}) \% m == 0$. Implement `compute_jaccard_modimizer.py` that extends your existing `compute_jaccard.py` code to only store and evaluate the modimizers in the sequences. The level of sampling should be set with the `-m` parameter. Also

output the number of modimizers found in each sequence. Note the default hash function in python is salted (it changes with every run), so instead use a stable function with zlib32 crc: `hash = zlib.crc32(kmer.encode('utf-8')) & 0xffffffff`

```
#compute_jaccard_modimizer.py
import zlib

def build_kmer_set(input_file, k_size, mod):

    fastastart = input_file.readline() #throw out first line
    if (not fastastart.startswith(">")):
        input_file.seek(0)

    kmer_set = set()
    prev_k = "" #maintain k-1 previous characters to build kmer

    for line in input_file:
        line = list(line.strip().upper())

        for m in range(len(line)):
            if (line[m] not in ['A', 'C', 'G', 'T']):
                line[m] = 'N'

        line = ''.join(line)

        sequence = prev_k + line

        #Build k-mers and add to dictionary
        for i in range(len(sequence) - k_size + 1):

            kmer = sequence[i:i+k_size]

            hash = zlib.crc32(kmer.encode('utf-8')) & 0xffffffff

            if hash % mod == 0:
                kmer_set.add(kmer)

        #Overlap
        prev_k = sequence[-(k_size-1):]

    return kmer_set
```

```

import argparse
import math

#Need parser to allow for CLI flags
parser = argparse.ArgumentParser(description="Jaccard.py argument parser")

parser.add_argument('-a', '--sequence1', type=str, help="Sequence 1 to compare")
parser.add_argument('-b', '--sequence2', type=str, help="Sequence 2 to compare")
parser.add_argument('-k', '--kmer_length', type=int, help="length of kmer to compute Jaccard coefficient")
parser.add_argument('-m', '--mod', type=int, help="level of sampling to use")

args = parser.parse_args()

k = args.kmer_length
m = args.mod

#Open required files
input_file1 = open(args.sequence1)
input_file2 = open(args.sequence2)

#Build kmer index for sequence 1 and sequence 2
kmer_set_1 = build_kmer_set(input_file1, k, m)
num_mod_1 = len(kmer_set_1)
kmer_set_2 = build_kmer_set(input_file2, k, m)
num_mod_2 = len(kmer_set_2)

#Calculate and display outputs
union = kmer_set_1.union(kmer_set_2)
intersection = kmer_set_1.intersection(kmer_set_2)

if not union:
    raise ValueError(
        "No sampled k-mers were found. Check the input FASTA files, "
        "k-mer length, and sampling modulus."
    )

jaccard = len(intersection) / len(union)
ani = ((2*jaccard) / (1 + jaccard)) ** (1/k)
ani_approx = 1 + (1/k)*math.log((2*jaccard)/(1 + jaccard))
print(f"The Jaccard coefficient is {jaccard:.4f}.")

```

```

print(f"The Average Nucleotide Identity is exactly {ani:.4f}.")
print(f"The Average Nucleotide Identity is approximately {ani_approx:.4f}.")
print(f"The number of modimizers found in sequence 1 is {num_mod_1} and the number found in sequence 2 is {num_mod_2}.")

```

```

input_file1.close()
input_file2.close()

```

Q1f. Now run your compute_jaccard_modimizers.py program using m=100 and m=1000 to output the jaccard, exact ANI, approximate ANI, and the number of modimizers found in each sequence. Comment on how well the modimizers approximate ANI values as found in Q1d as well as the amount of space needed to reach this level.

#Made by ChatGPT to print markdown table

```

import glob
import re
from pathlib import Path

```

```

results = []

```

```

# Look for files such as: modimizers_mut0.01_s1_m1000.txt
files = glob.glob("modimizers_mut*.txt")

```

```

for filename in files:
    # Get only the filename without its extension
    name = Path(filename).stem

    # Expected format:
    # modimizers_mut0.01_s1_m1000
    match = re.fullmatch(
        r"modimizers_mut([0-9]+(?:\.[0-9]+)?)_([0-9]+)_m([0-9]+)",
        name
    )

```

```

if match is None:
    print("Skipping:", filename)
    continue

```

```

mutation_proportion = float(match.group(1))
seed = match.group(2)
m_value = int(match.group(3))

```

```

with open(filename, "r") as f:
    lines = f.readlines()

```

```

if len(lines) < 4:
    print("Skipping incomplete file:", filename)
    continue

# Extract values from the first three lines
jaccard = float(lines[0].split()[-1].rstrip("."))
ani_exact = float(lines[1].split()[-1].rstrip("."))
ani_approx = float(lines[2].split()[-1].rstrip("."))

# Extract modimizer counts from the fourth line
modimizer_match = re.search(
    r"sequence 1 is (\d+) and the number found in sequence 2 is (\d+)",
    lines[3]
)

if modimizer_match is None:
    print("Could not extract modimizer counts from:", filename)
    continue

modimizers_sequence_1 = int(modimizer_match.group(1))
modimizers_sequence_2 = int(modimizer_match.group(2))

results.append([
    filename,
    mutation_proportion,
    seed,
    m_value,
    jaccard,
    ani_exact,
    ani_approx,
    modimizers_sequence_1,
    modimizers_sequence_2
])

# Sort by mutation proportion, then m value, then filename
results.sort(key=lambda x: (x[1], x[3], x[0]))

# Print Markdown table
print(
    "| Filename | Expected Mutation Proportion | Seed | m | "
    "Jaccard Coefficient | ANI Exact | ANI Approximate | "

```

```

    "Modimizers in Sequence 1 | Modimizers in Sequence 2 |"
)
print("---|---:---|---:---:---:---:---:---:|")

for (
    filename,
    mutation_proportion,
    seed,
    m_value,
    jaccard,
    ani_exact,
    ani_approx,
    modimizers_sequence_1,
    modimizers_sequence_2
) in results:
    print(
        f"| {filename} | "
        f"{mutation_proportion:.4f} | "
        f"{seed} | "
        f"{m_value} | "
        f"{jaccard:.6f} | "
        f"{ani_exact:.6f} | "
        f"{ani_approx:.6f} | "
        f"{modimizers_sequence_1} | "
        f"{modimizers_sequence_2} |"
    )

print(f"\nProcessed {len(results)} files.")

| Filename | Expected Mutation Proportion | Seed | m | Jaccard Coefficient | ANI Exact | ANI Approximate | Modimizers in Sequence 1 | Modimizers in Sequence 2 |
|---|---:---|---:---:---:---:---:---:|
| modimizers_mut0.01_s1_m100.txt | 0.0100 | s1 | 100 | 0.675100 | 0.989800 | 0.989700 | 9290 | 9521 |
| modimizers_mut0.01_s2_m100.txt | 0.0100 | s2 | 100 | 0.682300 | 0.990100 | 0.990000 | 9290 | 9464 |
| modimizers_mut0.01_s3_m100.txt | 0.0100 | s3 | 100 | 0.679900 | 0.990000 | 0.989900 | 9290 | 9384 |
| modimizers_mut0.01_s1_m1000.txt | 0.0100 | s1 | 1000 | 0.675200 | 0.989800 | 0.989700 | 931 | 957 |
| modimizers_mut0.01_s2_m1000.txt | 0.0100 | s2 | 1000 | 0.668800 | 0.989500 | 0.989500 | 931 | 933 |
| modimizers_mut0.01_s3_m1000.txt | 0.0100 | s3 | 1000 | 0.685300 | 0.990200 | 0.990200 | 931 | 970 |
| modimizers_mut0.05_s1_m100.txt | 0.0500 | s1 | 100 | 0.209300 | 0.950700 | 0.949500 | 9290 | 9775 |
| modimizers_mut0.05_s2_m100.txt | 0.0500 | s2 | 100 | 0.212700 | 0.951300 | 0.950100 | 9290 | 9912 |
| modimizers_mut0.05_s3_m100.txt | 0.0500 | s3 | 100 | 0.203000 | 0.949600 | 0.948300 | 9290 | 9839 |
| modimizers_mut0.05_s1_m1000.txt | 0.0500 | s1 | 1000 | 0.211900 | 0.951200 | 0.950000 | 931 | 922 |
| modimizers_mut0.05_s2_m1000.txt | 0.0500 | s2 | 1000 | 0.207700 | 0.950400 | 0.949200 | 931 | 1052 |

```

modimizers_mut0.05_s3_m1000.txt	0.0500	s3	1000	0.209500	0.950800	0.949500	931	957
modimizers_mut0.10_s1_m100.txt	0.1000	s1	100	0.063710	0.903900	0.898900	9290	9945
modimizers_mut0.10_s2_m100.txt	0.1000	s2	100	0.061850	0.902700	0.897600	9290	9922
modimizers_mut0.10_s3_m100.txt	0.1000	s3	100	0.063060	0.903500	0.898500	9290	9927
modimizers_mut0.10_s1_m1000.txt	0.1000	s1	1000	0.066230	0.905500	0.900700	931	1017
modimizers_mut0.10_s2_m1000.txt	0.1000	s2	1000	0.068870	0.907000	0.902400	931	1009
modimizers_mut0.10_s3_m1000.txt	0.1000	s3	1000	0.056840	0.899300	0.893800	931	984
modimizers_mut0.15_s1_m100.txt	0.1500	s1	100	0.020200	0.857500	0.846300	9290	9948
modimizers_mut0.15_s2_m100.txt	0.1500	s2	100	0.018180	0.853300	0.841300	9290	9916
modimizers_mut0.15_s3_m100.txt	0.1500	s3	100	0.018720	0.854400	0.842700	9290	9971
modimizers_mut0.15_s1_m1000.txt	0.1500	s1	1000	0.018690	0.854400	0.842600	931	977
modimizers_mut0.15_s2_m1000.txt	0.1500	s2	1000	0.026660	0.868600	0.859200	931	1033
modimizers_mut0.15_s3_m1000.txt	0.1500	s3	1000	0.015980	0.848100	0.835300	931	1040

Processed 24 files.

Answer: The modimizers approximate the ANI very well for both approximate and exact. They are especially close for lower levels of mutation. With an expected mutation proportion of 0.01, the ANIs are nearly indistinguishable. At higher levels, like 0.15, the difference jumps to a max of around 0.015, which is 0.006 more than the max from Q1d. However, this is still a very small difference. This approximation is sufficiently accurate, even with $m = 1000$. The numbers are more accurate at $m = 100$, but with the level of mutation being 0.15 at most, the difference is not significant. One thing to note is that with higher values of m , there is more instability in the calculations, meaning they differ a lot more depending on the seed.

Question 2. de Bruijn Graph construction Q2a. Write a script (in python, R, C++, etc) to draw the de Bruijn graph for the following reads using $k=3$ (assume all reads are from the forward strand, no sequencing errors, complete coverage of the genome). ATTCA ATTGA CATTG CTTAT GATTG TATTT TCATT TCTTA TGATT TTATT TTCAT TTCTT TTGAT.

```
text_string = """ATTCA
ATTGA
CATTG
CTTAT
GATTG
TATTT
TCATT
TCTTA
TGATT
TTATT
TTCAT
TTCTT
TTGAT""".split()

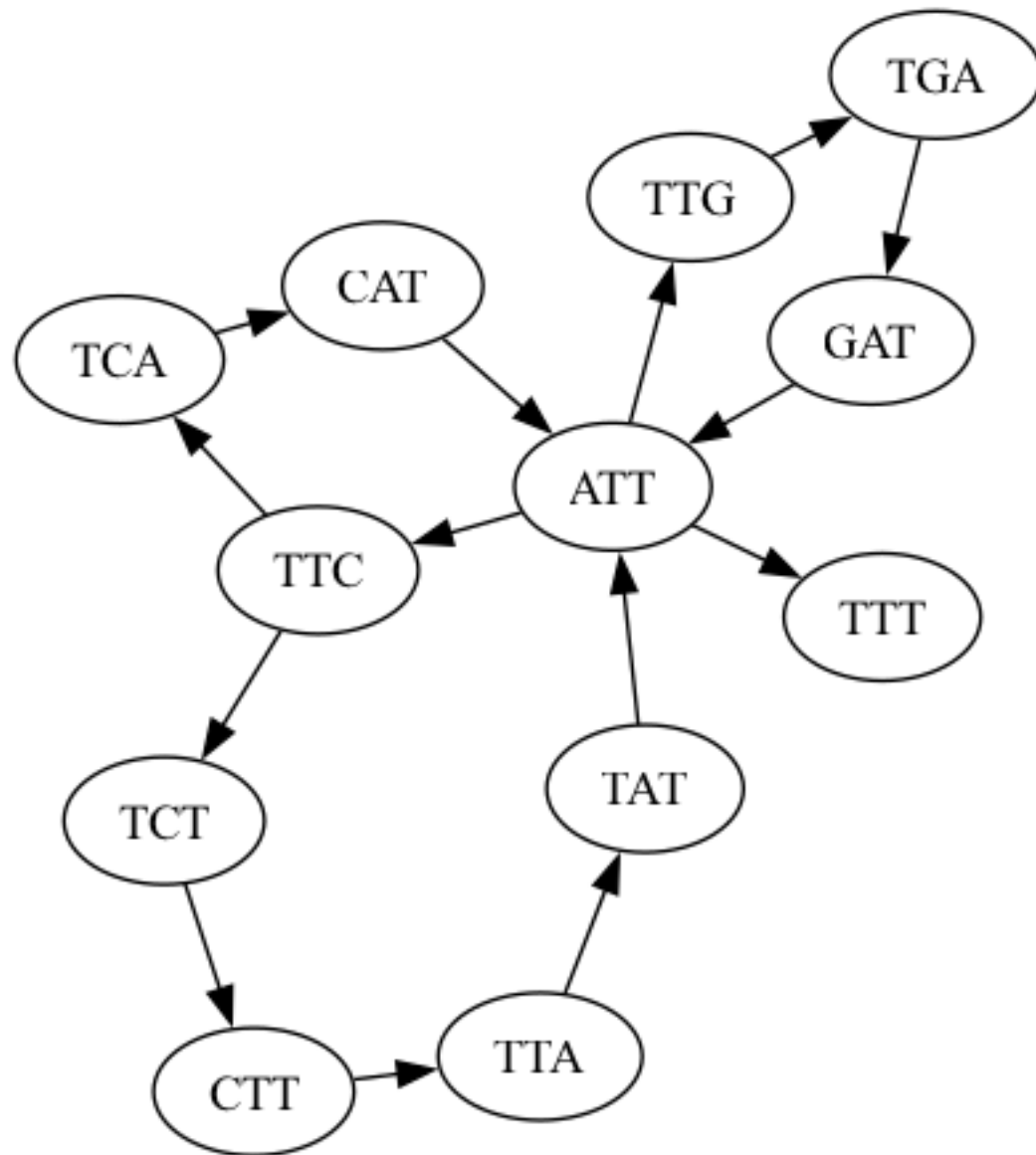
print(text_string)
output_file = open("graph.dot", 'w')
output_file.write("strict digraph{\n\n")
```

```
k = 3
for kmer in text_string:
    length = len(kmer)

    for i in range(length - k):
        output_file.write(f"    {kmer[i:i+3]} -> {kmer[i+1:i+4]}\n")

output_file.write("}")
output_file.close()

['ATTCA', 'ATTGA', 'CATTG', 'CTTAT', 'GATTG', 'TATTT', 'TCATT', 'TCTTA', 'TGATT', 'TTATT', 'TTCAT', 'TTCTT', 'TTGAT']
```



Q2b. Assume that the maximum number of occurrences of any 3-mer in the actual genome is 4 using the k-mers from Q2a. Write one possible genome sequence. Note this genome sequence may include sequences that are not supported by the underlying reads (this is the well known read decoherency problem for genome assembly).

Answer (possible): TCA CAT ATT TTG TGA GAT ATT TTC TCT CTT TTA TAT ATT TCC TCA CAT ATT TTT

Q2c. What would it take to fully resolve the genome? [In a few sentences]

Answer: Fully resolving the genome would require acquiring information about the exact number of times the kmer shows up in the genome and/or information about how far apart kmers are. Looking at kmer coverage could be a good indicator of the number of times the graph should be passing through the kmer. In addition, using a method like Illumina paired-ends sequencing could allow us to identify the amount of separation between two kmers, allowing us to better predict the actual number of times to go around loops in the graph. These are just two of the pieces we need to put together the genome puzzle since it's necessary to determine how the graph is meant to be linked together.

Question 3: Kmer Analysis Use `jellyfish` to count the 21-mers in the reads data (all reads should be counted together). Make sure to use the `"-C"` flag to count canonical kmers, otherwise your analysis will not correctly account for the fact that your reads come from either strand of DNA.

Question 3a. How many kmers occur exactly 50 times?

Answer: 932 kmers occur 50 times.

Question 3b. What are the top 10 most frequently occurring kmers?

#Used Claude to somehow get the commands from terminal to work in a notebook. I did it in terminal first.

```
import glob
import subprocess

subprocess.run(["jellyfish", "count", "-m", "21", "-o", "mer_counts.jf", "-C", "-s", "1000000",
               *glob.glob("./asm/*.fq")], check=True)

subprocess.run(["jellyfish", "merge", "-o", "mer_counts.jf", *glob.glob("mer_counts.jf_*")], check=True)

with open("reads.histo", "w") as fh:
    subprocess.run(["jellyfish", "histo", "mer_counts.jf"], stdout=fh, check=True)

print("Kmer:                Count:")
subprocess.run("jellyfish dump -c mer_counts.jf | sort -k2,2nr | head -n 10", shell=True, check=True)

Kmer:                Count:
AGGTTCAATTCCTGCCGGGCG 93
CGCGCCCGGCAGGAATTGAAC 91
GCGCCCGGCAGGAATTGAACC 91
CAGGTTCAATTCCTGCCGGGC 90
CCCGGCAGGAATTGAACCTGC 90
CCGGCAGGAATTGAACCTGCG 90
GAAGGTCGCAGGTTCAATTCC 90
GCAGGAATTGAACCTGCGACC 90
GCGCGCCCGGCAGGAATTGAA 90
GGCGCGCCCGGCAGGAATTGA 90
```

```
CompletedProcess(args='jellyfish dump -c mer_counts.jf | sort -k2,2nr | head -n 10', returncode=0)
```

Question 3c. What is the estimated genome size based on the kmer frequencies?

Answer: 233,492 bp

Question 3d. How well does the GenomeScope genome size estimate compare to the reference genome?

```
with open("asm/ref.fa") as file:
    print(sum(len(line.strip()) for line in file if not line.startswith(">")))
```

233806

Answer: The GenomeScope genome size estimate is very close to the size of the reference genome. It only differs by about 300 or so bases, which is an underestimate by only 0.1%.

Question 4. De Novo Assembly Question 4a. How many contigs were produced?

```
import subprocess
import os
```

```
os.chdir("/Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/asm/")
subprocess.run([
    "spades.py",
    "--isolate",
    "--pe1-1", "frag180.1.fq",
    "--pe1-2", "frag180.2.fq",
    "--mp1-1", "jump2k.1.fq",
    "--mp1-2", "jump2k.2.fq",
    "-o", "asm",
    "-t", "4",
    "-k", "31",
], check=True)
```

```
== Warning ==  output dir is not empty! Please, clean output directory before run.
```

Command line: /Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/miniforge3/envs/asn2/bin/spades.py --isolate --pe1-1 /Users/ashrithkol

System information:

SPAdes version: 4.3.0

Python version: 3.14.7

OS: macOS-26.4.1-arm64-arm-64bit-Mach-O

Output dir: /Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/asm/asm
Mode: ONLY assembling (without read error correction)
Debug mode is turned OFF

Dataset parameters:

Isolate mode

Reads:

Library number: 1, library type: paired-end

orientation: fr

left reads: ['/Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/asm/frag180.1.fq']

right reads: ['/Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/asm/frag180.2.fq']

interlaced reads: not specified

single reads: not specified

merged reads: not specified

Library number: 2, library type: mate-pairs

orientation: rf

left reads: ['/Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/asm/jump2k.1.fq']

right reads: ['/Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/asm/jump2k.2.fq']

interlaced reads: not specified

single reads: not specified

merged reads: not specified

Assembly parameters:

k: [31]

Repeat resolution is enabled

Mismatch careful mode is turned OFF

MismatchCorrector will be SKIPPED

Coverage cutoff is turned OFF

Assembly graph output will use GFA v1.2 format

Other parameters:

Dir for temp files: /Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/asm/asm/tmp

Threads: 4

Memory limit (in Gb): 250

===== SPAdes pipeline started. Log can be found here: /Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/asm/asm/spades.1

/Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/asm/frag180.1.fq: max reads length: 100

/Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/asm/frag180.2.fq: max reads length: 100

Reads length: 100

==== Before start started.

==== Assembling started.

==== K31 started.

== Running: /Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/miniforge3/envs/asn2/bin/spades-core /Users/ashrithkollu/Documents/Class

0:00:00.000	1M / 5M	INFO	General	(main.cpp	: 94)	Loaded config from "/Users/ashrithkollu/Docu
0:00:00.001	1M / 5M	INFO	General	(main.cpp	: 94)	Loaded config from "/Users/ashrithkollu/Docu
0:00:00.001	1M / 5M	WARN	General	(memory_limit.cpp	: 53)	Failed to limit memory to 250 Gb, setrlimit(
0:00:00.001	1M / 5M	INFO	General	(main.cpp	: 102)	Starting SPAdes, built from N/A, git revisio
0:00:00.001	1M / 5M	INFO	General	(main.cpp	: 103)	Maximum k-mer length: 128
0:00:00.001	1M / 5M	INFO	General	(main.cpp	: 104)	Assembling dataset ("/Users/ashrithkollu/Doc
0:00:00.001	1M / 5M	INFO	General	(main.cpp	: 105)	Maximum # of threads to use (adjusted due to
0:00:00.002	1M / 5M	INFO	General	(pipeline.cpp	: 221)	SPAdes started
0:00:00.002	1M / 5M	INFO	General	(pipeline.cpp	: 234)	Starting from stage: read_conversion
0:00:00.002	1M / 5M	INFO	General	(pipeline.cpp	: 245)	Two-step repeat resolution disabled
0:00:00.002	1M / 6M	INFO	GraphCore	(graph_core.hpp	: 582)	Graph created, vertex min_id: 3, edge min_id
0:00:00.002	1M / 6M	INFO	GraphCore	(graph_core.hpp	: 583)	Vertex size: 48, edge size: 40
0:00:00.003	1M / 6M	INFO	General	(edge_index.hpp	: 134)	Size of edge index entries: 12/8
0:00:00.004	1M / 6M	INFO	General	(pipeline.cpp	: 256)	Will need read mapping, kmer mapper will be
0:00:00.004	1M / 6M	INFO	StageManager	(stage.cpp	: 210)	STAGE == Binary Read Conversion (id: read_co
0:00:00.005	1M / 6M	INFO	General	(read_converter.cpp	: 78)	Converting reads to binary format for librari
0:00:00.005	1M / 6M	INFO	General	(read_converter.cpp	: 99)	Converting paired reads
0:00:00.047	63M / 63M	INFO	General	(binary_converter.cpp	: 127)	16384 reads processed
0:00:00.052	63M / 63M	INFO	General	(binary_converter.cpp	: 127)	32768 reads processed
0:00:00.077	63M / 63M	INFO	General	(binary_converter.cpp	: 143)	35192 reads written
0:00:00.079	33M / 42M	INFO	General	(read_converter.cpp	: 113)	Converting single reads
0:00:00.080	43M / 43M	INFO	General	(binary_converter.cpp	: 143)	0 reads written
0:00:00.081	33M / 42M	INFO	General	(read_converter.cpp	: 119)	Converting merged reads
0:00:00.082	43M / 43M	INFO	General	(binary_converter.cpp	: 143)	0 reads written
0:00:00.083	0M / 42M	INFO	General	(read_converter.cpp	: 78)	Converting reads to binary format for librari
0:00:00.083	0M / 42M	INFO	General	(read_converter.cpp	: 99)	Converting paired reads
0:00:00.125	55M / 55M	INFO	General	(binary_converter.cpp	: 127)	16384 reads processed
0:00:00.128	55M / 55M	INFO	General	(binary_converter.cpp	: 127)	32768 reads processed
0:00:00.133	55M / 55M	INFO	General	(binary_converter.cpp	: 127)	65536 reads processed
0:00:00.163	55M / 55M	INFO	General	(binary_converter.cpp	: 143)	70384 reads written
0:00:00.165	25M / 54M	INFO	General	(read_converter.cpp	: 113)	Converting single reads

0:00:00.166	35M / 54M	INFO	General	(binary_converter.cpp	: 143)	0 reads written
0:00:00.166	25M / 54M	INFO	General	(read_converter.cpp	: 119)	Converting merged reads
0:00:00.167	35M / 54M	INFO	General	(binary_converter.cpp	: 143)	0 reads written
0:00:00.200	1M / 54M	INFO	StageManager	(stage.cpp	: 210)	STAGE == de Bruijn graph construction (id: c
0:00:00.201	1M / 54M	INFO	General	(construction.cpp	: 150)	Max read length 100
0:00:00.201	1M / 54M	INFO	General	(construction.cpp	: 156)	Average read length 100
0:00:00.201	1M / 54M	INFO	General	(stage.cpp	: 120)	PROCEDURE == k+1-mer counting (id: construct
0:00:00.201	1M / 54M	INFO	General	(kmer_index_builder.hpp	: 308)	Splitting kmer instances into 40 files using
0:00:00.203	1M / 54M	INFO	General	(file_limit.hpp	: 43)	Open file limit set to 10240
0:00:00.204	1M / 54M	INFO	General	(kmer_splitter.hpp	: 96)	Memory available for splitting buffers: 7.15
0:00:00.204	1M / 54M	INFO	General	(kmer_splitter.hpp	: 104)	Using cell size of 1677721
0:00:00.315	2401M / 2401M	INFO	General	(kmerSplitters.hpp	: 128)	Processed 140768 reads
0:00:00.315	1M / 97M	INFO	General	(kmerSplitters.hpp	: 134)	Used 140768 reads
0:00:00.319	1M / 97M	INFO	General	(kmer_index_builder.hpp	: 314)	Starting k-mer counting.
0:00:00.330	1M / 98M	INFO	General	(kmer_index_builder.hpp	: 325)	K-mer counting done. There are 1314669 kmers
0:00:00.330	1M / 98M	INFO	General	(stage.cpp	: 120)	PROCEDURE == Extension index construction (i
0:00:00.331	1M / 98M	INFO	K-mer Index Building	(kmer_index_builder.hpp	: 503)	Building kmer index
0:00:00.331	1M / 98M	INFO	General	(kmer_index_builder.hpp	: 308)	Splitting kmer instances into 40 files using
0:00:00.334	1M / 98M	INFO	General	(file_limit.hpp	: 43)	Open file limit set to 10240
0:00:00.334	1M / 98M	INFO	General	(kmer_splitter.hpp	: 96)	Memory available for splitting buffers: 7.15
0:00:00.334	1M / 98M	INFO	General	(kmer_splitter.hpp	: 104)	Using cell size of 1677721
0:00:00.392	2401M / 2401M	INFO	General	(kmerSplitters.hpp	: 197)	Processed 1314669 kmers
0:00:00.392	2401M / 2401M	INFO	General	(kmerSplitters.hpp	: 202)	Used 1314669 kmers.
0:00:00.392	1M / 108M	INFO	General	(kmer_index_builder.hpp	: 314)	Starting k-mer counting.
0:00:00.409	1M / 107M	INFO	General	(kmer_index_builder.hpp	: 325)	K-mer counting done. There are 1300122 kmers
0:00:00.409	1M / 107M	INFO	K-mer Index Building	(kmer_index_builder.hpp	: 460)	Building perfect hash indices
0:00:00.435	2M / 106M	INFO	K-mer Index Building	(kmer_index_builder.hpp	: 496)	Index built. Total 1300122 kmers, 969096 byt
0:00:00.435	2M / 106M	INFO	General	(kmer_index_builder.hpp	: 191)	Merging final buckets.
0:00:00.454	3M / 107M	INFO	DeBruijnExtensionIndexBu	(kmer_extension_index_build	: 101)	Building k-mer extensions from k+1-mers
0:00:00.493	3M / 107M	INFO	DeBruijnExtensionIndexBu	(kmer_extension_index_build	: 106)	Building k-mer extensions from k+1-mers fini
0:00:00.493	3M / 97M	INFO	General	(stage.cpp	: 120)	PROCEDURE == Condensing graph (id: construct
0:00:00.494	3M / 97M	INFO	UnbranchingPathExtractor	(debruijn_graph_constructor	: 324)	Extracting unbranching paths
0:00:00.568	5M / 111M	INFO	UnbranchingPathExtractor	(debruijn_graph_constructor	: 348)	Extracting unbranching paths finished. 95967
0:00:00.618	5M / 102M	INFO	UnbranchingPathExtractor	(debruijn_graph_constructor	: 362)	Collecting perfect loops
0:00:00.639	5M / 112M	INFO	UnbranchingPathExtractor	(debruijn_graph_constructor	: 395)	Collecting perfect loops finished. 0 loops c
0:00:00.639	5M / 102M	INFO	DeBruijnGraphConstructor	(debruijn_graph_constructor	: 597)	Sorting edges...
0:00:00.650	5M / 102M	INFO	DeBruijnGraphConstructor	(debruijn_graph_constructor	: 602)	Edges sorted
0:00:00.650	5M / 102M	INFO	General	(debruijn_graph_constructor	: 511)	Total 191934 edges to create
0:00:00.650	13M / 102M	INFO	General	(debruijn_graph_constructor	: 514)	Collecting link records
0:00:00.714	16M / 108M	INFO	General	(debruijn_graph_constructor	: 521)	Total short graph edges: 150056
0:00:00.714	16M / 108M	INFO	General	(debruijn_graph_constructor	: 523)	Ordering link records
0:00:00.729	16M / 108M	INFO	General	(debruijn_graph_constructor	: 529)	Sorting done

0:00:00.729	17M / 108M	INFO	General	(debruijn_graph_constructor: 542)	Sorting LinkRecords...
0:00:00.736	17M / 108M	INFO	General	(debruijn_graph_constructor: 548)	LinkRecords sorted
0:00:00.736	17M / 108M	INFO	General	(debruijn_graph_constructor: 550)	Total 81420 vertices to create
0:00:00.736	24M / 108M	INFO	General	(debruijn_graph_constructor: 553)	Connecting the graph
0:00:00.789	19M / 85M	INFO	General	(stage.cpp : 120)	PROCEDURE == Filling coverage indices (PHM)
0:00:00.789	19M / 85M	INFO	K-mer Index Building	(kmer_index_builder.hpp : 460)	Building perfect hash indices
0:00:00.822	20M / 86M	INFO	K-mer Index Building	(kmer_index_builder.hpp : 496)	Index built. Total 1314669 kmers, 979624 bytes
0:00:00.822	25M / 91M	INFO	General	(coverage_hash_map_builder.cpp : 49)	Collecting k-mer coverage information from r
0:00:00.934	25M / 91M	INFO	General	(construction.cpp : 400)	Filling coverage and flanking coverage from l
0:00:00.993	25M / 91M	INFO	General	(coverage_filling.hpp : 83)	Processed 191934 edges
0:00:01.000	16M / 91M	INFO	StageManager	(stage.cpp : 210)	STAGE == EC Threshold Finding (id: ec_thresh
0:00:01.001	16M / 91M	INFO	General	(kmer_coverage_model.cpp : 182)	Kmer coverage valley at: 2
0:00:01.001	16M / 91M	INFO	General	(kmer_coverage_model.cpp : 202)	K-mer histogram maximum: 15
0:00:01.001	16M / 91M	INFO	General	(kmer_coverage_model.cpp : 238)	Estimated median coverage: 16. Coverage mad:
0:00:01.001	16M / 91M	INFO	General	(kmer_coverage_model.cpp : 260)	Fitting coverage model
0:00:01.003	16M / 91M	INFO	General	(kmer_coverage_model.cpp : 296)	... iteration 2
0:00:01.006	16M / 91M	INFO	General	(kmer_coverage_model.cpp : 296)	... iteration 4
0:00:01.021	16M / 91M	INFO	General	(kmer_coverage_model.cpp : 296)	... iteration 8
0:00:01.047	16M / 91M	INFO	General	(kmer_coverage_model.cpp : 296)	... iteration 16
0:00:01.058	16M / 91M	INFO	General	(kmer_coverage_model.cpp : 310)	Fitted mean coverage: 16.0382. Fitted covera
0:00:01.058	16M / 91M	INFO	General	(kmer_coverage_model.cpp : 335)	Probability of erroneous kmer at valley: 0.8
0:00:01.058	16M / 91M	INFO	General	(kmer_coverage_model.cpp : 359)	Preliminary threshold calculated as: 4
0:00:01.058	16M / 91M	INFO	General	(kmer_coverage_model.cpp : 363)	Threshold adjusted to: 4
0:00:01.058	16M / 91M	INFO	General	(kmer_coverage_model.cpp : 376)	Estimated genome size (ignoring repeats): 23
0:00:01.058	16M / 91M	INFO	General	(genomic_info_filler.cpp : 56)	Mean coverage was calculated as 16.0382
0:00:01.058	16M / 91M	INFO	General	(genomic_info_filler.cpp : 71)	EC coverage threshold value was calculated a
0:00:01.058	16M / 91M	INFO	General	(genomic_info_filler.cpp : 72)	Trusted kmer low bound: 0
0:00:01.058	16M / 91M	INFO	StageManager	(stage.cpp : 210)	STAGE == Gap Closer (id: early_gapcloser)
0:00:01.062	30M / 91M	INFO	General	(edge_index.hpp : 134)	Size of edge index entries: 12/8
0:00:01.070	31M / 93M	INFO	General	(gap_closer.cpp : 117)	Total edges in tip neighborhood: 29207 out o
0:00:01.070	31M / 93M	INFO	General	(edge_index.hpp : 196)	Using small index (max_id = 192896)
0:00:01.079	31M / 93M	INFO	K-mer Index Building	(kmer_index_builder.hpp : 460)	Building perfect hash indices
0:00:01.129	32M / 61M	INFO	K-mer Index Building	(kmer_index_builder.hpp : 496)	Index built. Total 588017 kmers, 425448 byte
0:00:01.133	36M / 63M	INFO	General	(edge_index_builders.hpp : 266)	Collecting edge information from graph, this
0:00:01.146	36M / 63M	INFO	General	(sequence_mapper_notifier.h: 80)	Starting sequence mapping
0:00:01.296	36M / 63M	INFO	General	(sequence_mapper_notifier.h: 120)	Total 35192 reads processed
0:00:01.299	31M / 63M	INFO	General	(gap_closer.cpp : 505)	Initializing gap closer
0:00:01.299	31M / 63M	INFO	GapCloser	(gap_closer.cpp : 421)	Collecting gap candidates
0:00:01.302	31M / 63M	INFO	GapCloser	(gap_closer.cpp : 425)	Total 42 tips collected, total 44 connection
0:00:01.302	31M / 63M	INFO	GapCloser	(gap_closer.cpp : 448)	Closing short gaps complete: filled 1 gaps a
0:00:01.304	31M / 63M	INFO	General	(gap_closer.cpp : 510)	Gap closer done
0:00:01.305	16M / 63M	INFO	StageManager	(stage.cpp : 210)	STAGE == Raw Simplification (id: raw_simplif

0:00:01.305	16M / 63M	INFO	General	(simplification.cpp	: 129)	PROCEDURE == Initial cleaning
0:00:01.306	16M / 64M	INFO	General	(simplification.cpp	: 63)	Most init cleaning disabled on main iteration
0:00:01.306	16M / 64M	INFO	Simplification	(parallel_processing.hpp	: 168)	Running Self conjugate edge remover
0:00:01.307	16M / 64M	INFO	Simplification	(parallel_processing.hpp	: 171)	Self conjugate edge remover triggered 0 times
0:00:01.307	16M / 64M	INFO	StageManager	(stage.cpp	: 210)	STAGE == Simplification (id: simplification)
0:00:01.308	16M / 64M	INFO	General	(simplification.cpp	: 397)	Graph simplification started
0:00:01.308	16M / 64M	INFO	General	(graph_simplification.hpp	: 646)	Creating parallel br instance
0:00:01.308	16M / 64M	INFO	General	(simplification.cpp	: 402)	PROCEDURE == Simplification cycle, iteration
0:00:01.308	16M / 64M	INFO	Simplification	(parallel_processing.hpp	: 168)	Running Tip clipper
0:00:01.358	18M / 64M	INFO	Simplification	(parallel_processing.hpp	: 171)	Tip clipper triggered 28760 times
0:00:01.358	18M / 64M	INFO	Simplification	(parallel_processing.hpp	: 168)	Running Bulge remover
0:00:01.845	48M / 65M	INFO	Simplification	(parallel_processing.hpp	: 171)	Bulge remover triggered 14637 times
0:00:01.845	48M / 65M	INFO	Simplification	(parallel_processing.hpp	: 168)	Running Low coverage edge remover
0:00:01.845	48M / 65M	INFO	Simplification	(parallel_processing.hpp	: 171)	Low coverage edge remover triggered 0 times
0:00:01.845	48M / 65M	INFO	General	(simplification.cpp	: 402)	PROCEDURE == Simplification cycle, iteration
0:00:01.845	48M / 65M	INFO	Simplification	(parallel_processing.hpp	: 168)	Running Tip clipper
0:00:01.848	48M / 65M	INFO	Simplification	(parallel_processing.hpp	: 171)	Tip clipper triggered 52 times
0:00:01.848	48M / 65M	INFO	Simplification	(parallel_processing.hpp	: 168)	Running Bulge remover
0:00:01.848	48M / 65M	INFO	Simplification	(parallel_processing.hpp	: 171)	Bulge remover triggered 0 times
0:00:01.848	48M / 65M	INFO	Simplification	(parallel_processing.hpp	: 168)	Running Low coverage edge remover
0:00:01.848	48M / 65M	INFO	Simplification	(parallel_processing.hpp	: 171)	Low coverage edge remover triggered 0 times
0:00:01.848	48M / 65M	INFO	General	(simplification.cpp	: 402)	PROCEDURE == Simplification cycle, iteration
0:00:01.848	48M / 65M	INFO	Simplification	(parallel_processing.hpp	: 168)	Running Tip clipper
0:00:01.848	48M / 65M	INFO	Simplification	(parallel_processing.hpp	: 171)	Tip clipper triggered 0 times
0:00:01.848	48M / 65M	INFO	Simplification	(parallel_processing.hpp	: 168)	Running Bulge remover
0:00:01.848	48M / 65M	INFO	Simplification	(parallel_processing.hpp	: 171)	Bulge remover triggered 0 times
0:00:01.848	48M / 65M	INFO	Simplification	(parallel_processing.hpp	: 168)	Running Low coverage edge remover
0:00:01.849	48M / 65M	INFO	Simplification	(parallel_processing.hpp	: 171)	Low coverage edge remover triggered 25 times
0:00:01.849	48M / 65M	INFO	General	(simplification.cpp	: 402)	PROCEDURE == Simplification cycle, iteration
0:00:01.849	48M / 65M	INFO	Simplification	(parallel_processing.hpp	: 168)	Running Tip clipper
0:00:01.850	48M / 65M	INFO	Simplification	(parallel_processing.hpp	: 171)	Tip clipper triggered 3 times
0:00:01.850	48M / 65M	INFO	Simplification	(parallel_processing.hpp	: 168)	Running Bulge remover
0:00:01.851	48M / 65M	INFO	Simplification	(parallel_processing.hpp	: 171)	Bulge remover triggered 3 times
0:00:01.851	48M / 65M	INFO	Simplification	(parallel_processing.hpp	: 168)	Running Low coverage edge remover
0:00:01.851	48M / 65M	INFO	Simplification	(parallel_processing.hpp	: 171)	Low coverage edge remover triggered 0 times
0:00:01.851	48M / 65M	INFO	General	(simplification.cpp	: 402)	PROCEDURE == Simplification cycle, iteration
0:00:01.851	48M / 65M	INFO	Simplification	(parallel_processing.hpp	: 168)	Running Tip clipper
0:00:01.851	48M / 65M	INFO	Simplification	(parallel_processing.hpp	: 171)	Tip clipper triggered 0 times
0:00:01.851	48M / 65M	INFO	Simplification	(parallel_processing.hpp	: 168)	Running Bulge remover
0:00:01.851	48M / 65M	INFO	Simplification	(parallel_processing.hpp	: 171)	Bulge remover triggered 0 times
0:00:01.851	48M / 65M	INFO	Simplification	(parallel_processing.hpp	: 168)	Running Low coverage edge remover
0:00:01.851	48M / 65M	INFO	Simplification	(parallel_processing.hpp	: 171)	Low coverage edge remover triggered 1 times

[illegible]

0:00:01.853	47M / 65M	INFO	StageManager	(stage.cpp	: 210)	STAGE == Gap Closer (id: late_gapcloser)
0:00:01.858	60M / 78M	INFO	General	(edge_index.hpp	: 134)	Size of edge index entries: 12/8
0:00:01.858	60M / 78M	INFO	General	(gap_closer.cpp	: 117)	Total edges in tip neighborhood: 240 out of
0:00:01.858	60M / 78M	INFO	General	(edge_index.hpp	: 196)	Using small index (max_id = 192896)
0:00:01.863	60M / 78M	INFO	K-mer Index Building	(kmer_index_builder.hpp	: 460)	Building perfect hash indices
0:00:01.884	60M / 78M	INFO	K-mer Index Building	(kmer_index_builder.hpp	: 496)	Index built. Total 236663 kmers, 171688 byte
0:00:01.886	62M / 79M	INFO	General	(edge_index_builders.hpp	: 266)	Collecting edge information from graph, this
0:00:01.893	62M / 79M	INFO	General	(sequence_mapper_notifier.h:	80)	Starting sequence mapping
0:00:02.008	62M / 79M	INFO	General	(sequence_mapper_notifier.h:	120)	Total 35192 reads processed
0:00:02.010	60M / 79M	INFO	General	(gap_closer.cpp	: 505)	Initializing gap closer
0:00:02.010	60M / 79M	INFO	GapCloser	(gap_closer.cpp	: 421)	Collecting gap candidates
0:00:02.010	60M / 79M	INFO	GapCloser	(gap_closer.cpp	: 425)	Total 0 tips collected, total 0 connection c
0:00:02.010	60M / 79M	INFO	GapCloser	(gap_closer.cpp	: 448)	Closing short gaps complete: filled 0 gaps a
0:00:02.011	60M / 79M	INFO	General	(gap_closer.cpp	: 510)	Gap closer done
0:00:02.011	47M / 79M	INFO	StageManager	(stage.cpp	: 210)	STAGE == Simplification Cleanup (id: simplif
0:00:02.011	47M / 79M	INFO	General	(simplification.cpp	: 189)	PROCEDURE == Post simplification
0:00:02.012	47M / 79M	INFO	General	(graph_simplification.hpp	: 455)	Disconnection of relatively low covered edge
0:00:02.012	47M / 79M	INFO	General	(graph_simplification.hpp	: 494)	Complex tip clipping disabled
0:00:02.012	47M / 79M	INFO	General	(graph_simplification.hpp	: 646)	Creating parallel br instance
0:00:02.012	47M / 79M	INFO	Simplification	(parallel_processing.hpp	: 168)	Running Tip clipper
0:00:02.012	47M / 79M	INFO	Simplification	(parallel_processing.hpp	: 171)	Tip clipper triggered 0 times
0:00:02.012	47M / 79M	INFO	Simplification	(parallel_processing.hpp	: 168)	Running Bulge remover
0:00:02.013	47M / 79M	INFO	Simplification	(parallel_processing.hpp	: 171)	Bulge remover triggered 0 times
0:00:02.013	47M / 79M	INFO	Simplification	(parallel_processing.hpp	: 168)	Running Tip clipper
0:00:02.013	47M / 79M	INFO	Simplification	(parallel_processing.hpp	: 171)	Tip clipper triggered 0 times
0:00:02.013	47M / 79M	INFO	Simplification	(parallel_processing.hpp	: 168)	Running Bulge remover
0:00:02.013	47M / 79M	INFO	Simplification	(parallel_processing.hpp	: 171)	Bulge remover triggered 0 times
0:00:02.013	47M / 79M	INFO	General	(simplification.cpp	: 348)	Disrupting self-conjugate edges
0:00:02.014	47M / 79M	INFO	Simplification	(parallel_processing.hpp	: 168)	Running Removing isolated edges
0:00:02.014	47M / 79M	INFO	Simplification	(parallel_processing.hpp	: 171)	Removing isolated edges triggered 116 times
0:00:02.014	47M / 79M	INFO	General	(simplification.cpp	: 508)	After simplification:
0:00:02.014	47M / 79M	INFO	General	(simplification.cpp	: 509)	Average coverage = 18.1942
0:00:02.015	47M / 79M	INFO	General	(simplification.cpp	: 510)	Total length = 234423
0:00:02.015	47M / 79M	INFO	General	(simplification.cpp	: 511)	Median edge length: 47818
0:00:02.015	47M / 79M	INFO	General	(simplification.cpp	: 512)	Edges: 20
0:00:02.015	47M / 79M	INFO	General	(simplification.cpp	: 513)	Vertices: 24
0:00:02.015	47M / 79M	INFO	StageManager	(stage.cpp	: 210)	STAGE == Mismatch Correction (id: mismatch_c
0:00:02.015	47M / 79M	INFO	General	(graph_pack_helpers.cpp	: 43)	Index refill
0:00:02.015	47M / 79M	INFO	General	(edge_index.hpp	: 175)	Using small index (max_id = 192896)
0:00:02.022	47M / 79M	INFO	K-mer Index Building	(kmer_index_builder.hpp	: 460)	Building perfect hash indices
0:00:02.061	47M / 79M	INFO	K-mer Index Building	(kmer_index_builder.hpp	: 496)	Index built. Total 234423 kmers, 170056 byte
0:00:02.063	49M / 79M	INFO	General	(edge_index_builders.hpp	: 253)	Collecting edge information from graph, this

0:00:02.077	49M / 79M	INFO	General	(graph_pack_helpers.cpp : 55)	Normalizing k-mer map. Total 986036 kmers to
0:00:02.205	49M / 79M	INFO	General	(kmer_mapper.hpp : 87)	Total 5404 kmers with non-trivial roots
0:00:02.206	49M / 79M	INFO	General	(graph_pack_helpers.cpp : 57)	Normalizing done
0:00:02.206	49M / 79M	INFO	General	(mismatch_correction.cpp : 421)	Collect potential mismatches
0:00:02.333	49M / 80M	INFO	General	(mismatch_correction.cpp : 215)	Total 20 edges (out of 20) with 32792 potent
0:00:02.333	49M / 80M	INFO	General	(mismatch_correction.cpp : 423)	Potential mismatches collected
0:00:02.336	49M / 80M	INFO	General	(sequence_mapper_notifier.h: 80)	Starting sequence mapping
0:00:02.711	58M / 83M	INFO	General	(sequence_mapper_notifier.h: 120)	Total 140768 reads processed
0:00:02.715	51M / 83M	INFO	General	(mismatch_correction.cpp : 416)	All edges processed
0:00:02.715	49M / 83M	INFO	General	(mismatch_correction.cpp : 471)	Corrected 0 nucleotides
0:00:02.715	49M / 83M	INFO	StageManager	(stage.cpp : 210)	STAGE == Contig Output (id: contig_output)
0:00:02.715	49M / 83M	INFO	General	(contig_output.hpp : 20)	Outputting contigs to "/Users/ashrithkollu/D
0:00:02.717	49M / 83M	INFO	General	(contig_output_stage.cpp : 159)	Writing GFA graph to "/Users/ashrithkollu/Do
0:00:02.718	49M / 83M	INFO	StageManager	(stage.cpp : 210)	STAGE == Paired Information Counting (id: la
0:00:02.718	49M / 83M	INFO	General	(graph_pack_helpers.cpp : 55)	Normalizing k-mer map. Total 986036 kmers to
0:00:02.718	49M / 83M	INFO	General	(graph_pack_helpers.cpp : 57)	Normalizing done
0:00:02.718	49M / 83M	INFO	General	(pair_info_count.cpp : 165)	Min edge length for estimation: 47818
0:00:02.718	49M / 83M	INFO	General	(pair_info_count.cpp : 176)	Estimating insert size for library #0
0:00:02.719	49M / 83M	INFO	General	(pair_info_count.cpp : 41)	Selecting usual mapper
0:00:02.719	49M / 83M	INFO	General	(paired_info_utils.cpp : 102)	Estimating insert size (takes a while)
0:00:02.742	129M / 157M	INFO	General	(sequence_mapper_notifier.h: 80)	Starting sequence mapping
0:00:02.897	129M / 155M	INFO	General	(sequence_mapper_notifier.h: 120)	Total 35192 reads processed
0:00:02.937	129M / 155M	INFO	General	(paired_info_utils.cpp : 119)	Edge pairs: 56
0:00:02.937	129M / 155M	INFO	General	(paired_info_utils.cpp : 123)	20874 paired reads (59.3146% of all) aligned
0:00:02.938	49M / 155M	INFO	General	(pair_info_count.cpp : 200)	Insert size = 179.96, deviation = 19.8919,
0:00:02.938	49M / 155M	INFO	General	(pair_info_count.cpp : 211)	Filtering data for library #0
0:00:02.938	49M / 155M	INFO	General	(pair_info_count.cpp : 41)	Selecting usual mapper
0:00:02.940	49M / 155M	INFO	General	(sequence_mapper_notifier.h: 80)	Starting sequence mapping
0:00:03.048	49M / 155M	INFO	General	(sequence_mapper_notifier.h: 120)	Total 35192 reads processed
0:00:03.048	49M / 155M	INFO	General	(pair_info_count.cpp : 216)	Mapping library #0
0:00:03.048	49M / 155M	INFO	General	(pair_info_count.cpp : 218)	Mapping paired reads (takes a while)
0:00:03.048	49M / 155M	INFO	General	(pair_info_count.cpp : 41)	Selecting usual mapper
0:00:03.048	49M / 155M	INFO	General	(paired_info_utils.cpp : 155)	Left insert size quantile 155, right insert
0:00:03.050	62M / 163M	INFO	General	(sequence_mapper_notifier.h: 80)	Starting sequence mapping
0:00:03.167	62M / 163M	INFO	General	(sequence_mapper_notifier.h: 120)	Total 35192 reads processed
0:00:03.168	49M / 163M	INFO	General	(pair_info_count.cpp : 176)	Estimating insert size for library #1
0:00:03.169	49M / 163M	INFO	General	(pair_info_count.cpp : 32)	Mapping mate-pairs using BWA-mem mapper
0:00:03.170	49M / 163M	INFO	BWAIndex	(bwa_index.cpp : 42)	Setting BWA alignment mode to 'default'
0:00:03.172	49M / 163M	INFO	General	(bwa_index.cpp : 150)	Using BWA IS algorithm
0:00:03.208	49M / 163M	INFO	General	(paired_info_utils.cpp : 102)	Estimating insert size (takes a while)
0:00:03.215	129M / 203M	INFO	General	(sequence_mapper_notifier.h: 80)	Starting sequence mapping
0:00:03.569	129M / 203M	INFO	General	(sequence_mapper_notifier.h: 120)	Total 70384 reads processed

0:00:03.610	129M / 203M	INFO	General	(paired_info_utils.cpp : 119)	Edge pairs: 25
0:00:03.610	129M / 203M	INFO	General	(paired_info_utils.cpp : 123)	41191 paired reads (58.5232% of all) aligned
0:00:03.610	49M / 203M	INFO	General	(pair_info_count.cpp : 200)	Insert size = 2001.19, deviation = 200.689
0:00:03.611	49M / 203M	INFO	General	(pair_info_count.cpp : 216)	Mapping library #1
0:00:03.611	49M / 203M	INFO	General	(pair_info_count.cpp : 218)	Mapping paired reads (takes a while)
0:00:03.611	49M / 203M	INFO	General	(pair_info_count.cpp : 32)	Mapping mate-pairs using BWA-mem mapper
0:00:03.611	49M / 203M	INFO	BWAIndex	(bwa_index.cpp : 42)	Setting BWA alignment mode to 'default'
0:00:03.613	49M / 203M	INFO	General	(bwa_index.cpp : 150)	Using BWA IS algorithm
0:00:03.649	49M / 203M	INFO	General	(paired_info_utils.cpp : 155)	Left insert size quantile 1744, right insert
0:00:03.650	62M / 203M	INFO	General	(sequence_mapper_notifier.h: 80)	Starting sequence mapping
0:00:03.971	62M / 203M	INFO	General	(sequence_mapper_notifier.h: 120)	Total 70384 reads processed
0:00:03.972	49M / 203M	INFO	StageManager	(stage.cpp : 210)	STAGE == Distance Estimation (id: distance_e
0:00:03.973	49M / 203M	INFO	General	(distance_estimation.cpp : 75)	Processing library #0
0:00:03.974	49M / 203M	INFO	General	(distance_estimation_utils.: 23)	Estimating distances
0:00:03.974	49M / 203M	INFO	DistanceEstimator	(distance_estimation.hpp : 132)	Using SIMPLE distance estimator
0:00:03.978	49M / 203M	INFO	General	(distance_estimation_utils.: 137)	Refining clustered pair information
0:00:03.978	49M / 203M	INFO	General	(distance_estimation_utils.: 139)	The refining of clustered pair information h
0:00:03.978	49M / 203M	INFO	General	(distance_estimation_utils.: 141)	Improving paired information
0:00:03.982	49M / 203M	INFO	PairInfoImprover	(pair_info_improver.hpp : 81)	Paired info stats: missing = 4; contradictio
0:00:03.986	49M / 203M	INFO	PairInfoImprover	(pair_info_improver.hpp : 81)	Paired info stats: missing = 0; contradictio
0:00:03.986	49M / 203M	INFO	General	(distance_estimation_utils.: 81)	Filling scaffolding index
0:00:03.986	49M / 203M	INFO	General	(distance_estimation_utils.: 23)	Estimating distances
0:00:03.986	49M / 203M	INFO	DistanceEstimator	(distance_estimation.hpp : 132)	Using SMOOTHING distance estimator
0:00:03.987	49M / 203M	INFO	General	(distance_estimation.cpp : 84)	Clearing raw paired index
0:00:03.988	49M / 203M	INFO	StageManager	(stage.cpp : 210)	STAGE == Repeat Resolving (id: repeat_resolv
0:00:03.988	49M / 203M	INFO	General	(repeat_resolving.cpp : 88)	Using Path-Extend repeat resolving
0:00:03.988	49M / 203M	INFO	General	(launcher.cpp : 636)	ExSPAnder repeat resolving tool started
0:00:03.988	49M / 203M	INFO	General	(launcher.cpp : 193)	Autodetecting unique edge set parameters...
0:00:03.988	49M / 203M	INFO	General	(launcher.cpp : 195)	Minimal unique edge length set to the smalle
0:00:03.989	49M / 203M	INFO	General	(coverage_uniformity_analyz: 25)	genomic coverage is 18.1438 calculated of le
0:00:03.989	49M / 203M	INFO	General	(launcher.cpp : 201)	median coverage for edges longer than 2001 i
0:00:03.989	49M / 203M	INFO	ScaffoldingUniqueEdgeAna	(scaff_supplementary.cpp : 78)	With length cutoff: 2001, median long edge c
0:00:03.989	49M / 203M	INFO	ScaffoldingUniqueEdgeAna	(scaff_supplementary.cpp : 81)	Unique edges quantity: 12, unique edges leng
0:00:03.990	49M / 203M	INFO	General	(launcher.cpp : 444)	Creating main extenders, unique edge length
0:00:03.990	49M / 203M	INFO	General	(extenders_logic.cpp : 341)	Estimated coverage of library #0 is 18.1942
0:00:03.991	49M / 203M	INFO	General	(extenders_logic.cpp : 352)	Creating extender; library index size: 58
0:00:03.991	49M / 203M	INFO	General	(extenders_logic.cpp : 341)	Estimated coverage of library #0 is 18.1942
0:00:03.991	49M / 203M	INFO	General	(extenders_logic.cpp : 352)	Creating extender; library index size: 58
0:00:03.991	49M / 203M	INFO	General	(extenders_logic.cpp : 550)	Using 1 paired-end library
0:00:03.991	49M / 203M	INFO	General	(extenders_logic.cpp : 551)	Using 1 paired-end scaffolding library
0:00:03.991	49M / 203M	INFO	General	(extenders_logic.cpp : 552)	Using 0 single read libraries
0:00:03.991	49M / 203M	INFO	General	(launcher.cpp : 347)	Will add extenders for length 1701

0:00:03.992	49M / 203M	INFO	ScaffoldingUniqueEdgeAna	(scaff_supplementary.cpp	: 78)	With length cutoff: 1701, median long edge c
0:00:03.992	49M / 203M	INFO	ScaffoldingUniqueEdgeAna	(scaff_supplementary.cpp	: 81)	Unique edges quantity: 12, unique edges leng
0:00:03.992	49M / 203M	INFO	General	(launcher.cpp	: 347)	Will add extenders for length 1401
0:00:03.992	49M / 203M	INFO	ScaffoldingUniqueEdgeAna	(scaff_supplementary.cpp	: 78)	With length cutoff: 1401, median long edge c
0:00:03.992	49M / 203M	INFO	ScaffoldingUniqueEdgeAna	(scaff_supplementary.cpp	: 81)	Unique edges quantity: 12, unique edges leng
0:00:03.992	49M / 203M	INFO	General	(launcher.cpp	: 347)	Will add extenders for length 1101
0:00:03.992	49M / 203M	INFO	ScaffoldingUniqueEdgeAna	(scaff_supplementary.cpp	: 78)	With length cutoff: 1101, median long edge c
0:00:03.992	49M / 203M	INFO	ScaffoldingUniqueEdgeAna	(scaff_supplementary.cpp	: 81)	Unique edges quantity: 12, unique edges leng
0:00:03.992	49M / 203M	INFO	General	(launcher.cpp	: 347)	Will add extenders for length 801
0:00:03.993	49M / 203M	INFO	ScaffoldingUniqueEdgeAna	(scaff_supplementary.cpp	: 78)	With length cutoff: 801, median long edge co
0:00:03.993	49M / 203M	INFO	ScaffoldingUniqueEdgeAna	(scaff_supplementary.cpp	: 81)	Unique edges quantity: 12, unique edges leng
0:00:03.993	49M / 203M	INFO	General	(launcher.cpp	: 347)	Will add extenders for length 501
0:00:03.993	49M / 203M	INFO	ScaffoldingUniqueEdgeAna	(scaff_supplementary.cpp	: 78)	With length cutoff: 501, median long edge co
0:00:03.993	49M / 203M	INFO	ScaffoldingUniqueEdgeAna	(scaff_supplementary.cpp	: 81)	Unique edges quantity: 12, unique edges leng
0:00:03.993	49M / 203M	INFO	General	(launcher.cpp	: 352)	Will add final extenders for length 500
0:00:03.993	49M / 203M	INFO	ScaffoldingUniqueEdgeAna	(scaff_supplementary.cpp	: 78)	With length cutoff: 500, median long edge co
0:00:03.993	49M / 203M	INFO	ScaffoldingUniqueEdgeAna	(scaff_supplementary.cpp	: 81)	Unique edges quantity: 12, unique edges leng
0:00:03.993	49M / 203M	INFO	General	(extenders_logic.cpp	: 213)	Creating Scaffolding 2015 extender for lib #
0:00:03.993	49M / 203M	INFO	General	(extenders_logic.cpp	: 217)	Paired unclustered indices not empty, using
0:00:03.994	49M / 203M	INFO	ExtensionChooser2015	(extension_chooser2015.hpp	: 57)	ExtensionChooser2015 created
0:00:03.994	49M / 203M	INFO	General	(extenders_logic.cpp	: 397)	Using 1 mate-pair library
0:00:03.994	49M / 203M	INFO	General	(extenders_logic.cpp	: 213)	Creating Scaffolding 2015 extender for lib #
0:00:03.994	49M / 203M	INFO	General	(extenders_logic.cpp	: 217)	Paired unclustered indices not empty, using
0:00:03.994	49M / 203M	INFO	ExtensionChooser2015	(extension_chooser2015.hpp	: 57)	ExtensionChooser2015 created
0:00:03.994	49M / 203M	INFO	General	(extenders_logic.cpp	: 213)	Creating Scaffolding 2015 extender for lib #
0:00:03.994	49M / 203M	INFO	General	(extenders_logic.cpp	: 217)	Paired unclustered indices not empty, using
0:00:03.994	49M / 203M	INFO	ExtensionChooser2015	(extension_chooser2015.hpp	: 57)	ExtensionChooser2015 created
0:00:03.994	49M / 203M	INFO	General	(extenders_logic.cpp	: 213)	Creating Scaffolding 2015 extender for lib #
0:00:03.994	49M / 203M	INFO	General	(extenders_logic.cpp	: 217)	Paired unclustered indices not empty, using
0:00:03.994	49M / 203M	INFO	ExtensionChooser2015	(extension_chooser2015.hpp	: 57)	ExtensionChooser2015 created
0:00:03.994	49M / 203M	INFO	General	(extenders_logic.cpp	: 213)	Creating Scaffolding 2015 extender for lib #
0:00:03.994	49M / 203M	INFO	General	(extenders_logic.cpp	: 217)	Paired unclustered indices not empty, using
0:00:03.995	49M / 203M	INFO	ExtensionChooser2015	(extension_chooser2015.hpp	: 57)	ExtensionChooser2015 created
0:00:03.995	49M / 203M	INFO	General	(extenders_logic.cpp	: 213)	Creating Scaffolding 2015 extender for lib #
0:00:03.995	49M / 203M	INFO	General	(extenders_logic.cpp	: 217)	Paired unclustered indices not empty, using
0:00:03.995	49M / 203M	INFO	ExtensionChooser2015	(extension_chooser2015.hpp	: 57)	ExtensionChooser2015 created
0:00:03.995	49M / 203M	INFO	General	(extenders_logic.cpp	: 213)	Creating Scaffolding 2015 extender for lib #
0:00:03.995	49M / 203M	INFO	General	(extenders_logic.cpp	: 217)	Paired unclustered indices not empty, using
0:00:03.995	49M / 203M	INFO	ExtensionChooser2015	(extension_chooser2015.hpp	: 57)	ExtensionChooser2015 created
0:00:03.995	49M / 203M	INFO	General	(launcher.cpp	: 473)	Total number of extenders is 10
0:00:03.996	49M / 203M	INFO	General	(launcher.cpp	: 264)	Finalizing paths
0:00:03.996	49M / 203M	INFO	General	(launcher.cpp	: 266)	Deduplicating paths

0:00:03.996	49M / 203M	INFO	General	(launcher.cpp	: 270)	Paths deduplicated
0:00:03.996	49M / 203M	INFO	PEResolver	(pe_resolver.cpp	: 70)	Removing overlaps
0:00:03.996	49M / 203M	INFO	PEResolver	(pe_resolver.cpp	: 73)	Sorting paths
0:00:03.996	49M / 203M	INFO	PEResolver	(pe_resolver.cpp	: 80)	Marking overlaps
0:00:03.996	49M / 203M	INFO	OverlapRemover	(overlap_remover.hpp	: 120)	Marking start/end overlaps
0:00:03.996	49M / 203M	INFO	OverlapRemover	(overlap_remover.hpp	: 123)	Marking remaining overlaps
0:00:03.996	49M / 203M	INFO	PEResolver	(pe_resolver.cpp	: 83)	Splitting paths
0:00:03.996	49M / 203M	INFO	PEResolver	(pe_resolver.cpp	: 88)	Deduplicating paths
0:00:03.996	49M / 203M	INFO	PEResolver	(pe_resolver.cpp	: 90)	Overlaps removed
0:00:03.996	49M / 203M	INFO	General	(launcher.cpp	: 287)	Paths finalized
0:00:03.996	49M / 203M	INFO	General	(launcher.cpp	: 482)	Closing gaps in paths
0:00:03.997	49M / 203M	INFO	General	(launcher.cpp	: 512)	Gap closing completed
0:00:03.997	49M / 203M	INFO	General	(launcher.cpp	: 320)	Traversing tandem repeats
0:00:03.997	49M / 204M	INFO	General	(launcher.cpp	: 330)	Traversed 0 loops
0:00:03.997	49M / 204M	INFO	General	(launcher.cpp	: 264)	Finalizing paths
0:00:03.997	49M / 204M	INFO	General	(launcher.cpp	: 266)	Deduplicating paths
0:00:03.997	49M / 204M	INFO	General	(launcher.cpp	: 270)	Paths deduplicated
0:00:03.997	49M / 204M	INFO	PEResolver	(pe_resolver.cpp	: 70)	Removing overlaps
0:00:03.997	49M / 204M	INFO	PEResolver	(pe_resolver.cpp	: 73)	Sorting paths
0:00:03.997	49M / 204M	INFO	PEResolver	(pe_resolver.cpp	: 80)	Marking overlaps
0:00:03.997	49M / 204M	INFO	OverlapRemover	(overlap_remover.hpp	: 120)	Marking start/end overlaps
0:00:03.997	49M / 204M	INFO	OverlapRemover	(overlap_remover.hpp	: 123)	Marking remaining overlaps
0:00:03.997	49M / 204M	INFO	PEResolver	(pe_resolver.cpp	: 83)	Splitting paths
0:00:03.997	49M / 204M	INFO	PEResolver	(pe_resolver.cpp	: 88)	Deduplicating paths
0:00:03.997	49M / 204M	INFO	PEResolver	(pe_resolver.cpp	: 90)	Overlaps removed
0:00:03.998	49M / 204M	INFO	General	(launcher.cpp	: 287)	Paths finalized
0:00:03.998	49M / 204M	INFO	General	(launcher.cpp	: 701)	ExSPAnde repeat resolving tool finished
0:00:03.998	49M / 204M	INFO	StageManager	(stage.cpp	: 210)	STAGE == Contig Output (id: contig_output)
0:00:03.998	49M / 204M	INFO	General	(contig_output.hpp	: 20)	Outputting contigs to "/Users/ashrithkollu/D
0:00:03.999	49M / 204M	INFO	General	(contig_output_stage.cpp	: 159)	Writing GFA graph to "/Users/ashrithkollu/Do
0:00:04.000	49M / 204M	INFO	General	(contig_output_stage.cpp	: 175)	Outputting FastG graph to "/Users/ashrithkol
0:00:04.002	49M / 204M	INFO	General	(contig_output_stage.cpp	: 210)	Breaking scaffolds
0:00:04.003	49M / 204M	INFO	General	(contig_output_stage.cpp	: 101)	Outputting contigs to /Users/ashrithkollu/Do
0:00:04.004	49M / 204M	INFO	General	(contig_output_stage.cpp	: 107)	Outputting FastG paths to /Users/ashrithkoll
0:00:04.005	49M / 204M	INFO	General	(contig_output_stage.cpp	: 101)	Outputting contigs to /Users/ashrithkollu/Do
0:00:04.006	49M / 204M	INFO	General	(contig_output_stage.cpp	: 107)	Outputting FastG paths to /Users/ashrithkoll
0:00:04.006	49M / 204M	INFO	General	(contig_output_stage.cpp	: 114)	Populating GFA with scaffold paths
0:00:04.006	49M / 204M	INFO	General	(pipeline.cpp	: 305)	SPAdes finished
0:00:04.024	1M / 204M	INFO	General	(main.cpp	: 131)	Assembling time: 0 hours 0 minutes 4 seconds

===== K31 finished.

===== Copy files started.

== Running: /Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/miniforge3/envs/asn2/bin/python3 /Users/ashrithkollu/Documents/Classes/J

===== Copy files finished.

===== Assembling finished.

===== Breaking scaffolds started.

== Running: /Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/miniforge3/envs/asn2/bin/python3 /Users/ashrithkollu/Documents/Classes/J

===== Breaking scaffolds finished.

===== Terminate started.

===== Terminate finished.

- * Assembled contigs are in /Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/asm/asm/contigs.fasta
- * Assembled scaffolds are in /Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/asm/asm/scaffolds.fasta
- * Paths in the assembly graph corresponding to the contigs are in /Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/asm/a
- * Paths in the assembly graph corresponding to the scaffolds are in /Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/asm/a
- * Assembly graph is in /Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/asm/asm/assembly_graph.fastg
- * Assembly graph in GFA format is in /Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/asm/asm/assembly_graph_with_scaffo

===== SPAdes pipeline finished WITH WARNINGS!

=== Error correction and assembling warnings:

- * 0:00:00.001 1M / 5M WARN General (memory_limit.cpp : 53) Failed to limit memory to 250 Gb, setrlimit

===== Warnings saved to /Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/asm/asm/warnings.log

SPAdes log can be found here: /Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/asm/asm/spades.log

Thank you for using SPAdes! If you use it in your research, please cite:

Prjibelski, A., Antipov, D., Meleshko, D., Lapidus, A. and Korobeynikov, A., 2020. Using SPAdes de novo assembler. Current protocols in bioinformatics. doi.org/10.1002/cpbi.102

```
CompletedProcess(args=['spades.py', '--isolate', '--pe1-1', 'frag180.1.fq', '--pe1-2', 'frag180.2.fq', '--mp1-1', 'jump2k.1.fq', '--mp1-2', 'jump2k.2.fq'], returncode=0, stdout=b'4 contigs were produced\n', stderr=b'')\nos.chdir('/Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/asm/asm')\nresult = subprocess.run(['grep', '-c', '>', 'contigs.fasta'], capture_output=True, text=True)
```

```
print(f"{result.stdout.strip()} contigs were produced")
```

4 contigs were produced

Question 4b. What is the total length of the contigs?

```
subprocess.run(['samtools', 'faidx', 'contigs.fasta'], check=True)
```

```
length_array = []\nwith open("contigs.fasta.fai") as file:\n    for line in file:\n        contig, length, offset, line_bases, line_width = line.strip().split("\\t")\n        length_array.append(int(length))
```

```
print(f"Total contig length is {sum(length_array)}")
```

Total contig length is 234569

Question 4c. What is the size of your largest contig?

```
length_array.sort(reverse=True)\nprint(f"The largest contig is {length_array[0]} bp long")
```

The largest contig is 105834 bp long

Question 4d. What is the contig N50 size?

```
def find_N50(length_array):\n\n    n50_sum = 0\n\n    for value in length_array:\n        n50_sum += value\n\n    if n50_sum >= 1.0/2.0*sum(length_array):\n        return value
```

```

    return 0

print(f"The N50 size is {find_N50(length_array)}")
The N50 size is 47849

```

Question 5. Finding and decoding the insertion Use the mummersoftware tools to align your contigs to the reference genome. This will reveal all of the structural variants present, including the secret insertion.

Question 5a. How many insertions and deletions are in the assembly?

```

result = subprocess.run(['dnadiff', '/Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/asm/ref.fa', '/Users/ashrithkollu/D

print(result)

```

```
CompletedProcess(args=['dnadiff', '/Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/asm/ref.fa', '/Users/ashrithkollu/D
```

Answer: There are 6 indels in the assembly. There are 5 deletions (present in reference and not in query) spanning 45 bases and 1 insertions (present in query but not reference) spanning 808 bases.

Question 5b. What is the position of the insertion on the reference?

#Generated by ChatGPT with the prompt just asking to turn show-coords table into something I can put into a notebook

```

import subprocess
import pandas as pd
from io import StringIO

result = subprocess.run(
    ["show-coords", "out.delta"],
    capture_output=True,
    text=True
)

```

```

df = pd.read_csv(
    StringIO(result.stdout),
    sep=r"\s+",
    skiprows=5,
    header=None
)

```

```
display(df)
```

	0	1	2	3	4	5	6	7	8	9	10	\
0	127960	233793		105834	1		105834	105834		100.00		
1	221067	221165		6995	6897		99	99		95.96		

2	226799	226897		12727	12629		99	99		95.96	
3	40664	88512		47849	1		47849	47849		100.00	
4	3	26789		41445	14659		26787	26787		100.00	
5	26790	40639		13850	1		13850	13850		100.00	
6	88516	127956		39441	1		39441	39441		100.00	

	11	12
0	Halomonas	NODE_1_length_105834_cov_18.177840
1	Halomonas	NODE_1_length_105834_cov_18.177840
2	Halomonas	NODE_1_length_105834_cov_18.177840
3	Halomonas	NODE_2_length_47849_cov_17.929943
4	Halomonas	NODE_3_length_41445_cov_18.207587
5	Halomonas	NODE_3_length_41445_cov_18.207587
6	Halomonas	NODE_4_length_39441_cov_18.559959

Answer: There's a gap between base 26789 and 26790 in the reference.

Question 5c. How long is the novel insertion?

Answer: At this position, there is an insertion between base pairs 13850 and 14659, implying an insertion of 808 base pairs.

Question 5d. What is the DNA sequence of the encoded message?

```
import subprocess
```

```
contig = "NODE_3_length_41445_cov_18.207587"
start = 13851
end = 14658
```

```
result = subprocess.run(
    ["samtools", "faidx", "/Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/asm/asm/contigs.fasta", f"{contig}:{start}-{end}"],
    capture_output=True,
    text=True,
    check=True
)
```

```
print(result.stdout)
```

```
>NODE_3_length_41445_cov_18.207587:13851-14658
TAAATATGTAAATATTTACCGAGTCTGCCAGGCACTACTCTCGGACGAGTATAAGGGCA
TCCGCTTTGCAGGGGGCGTTACCTACTCTAGTAAGAGATGCAGTCTTAACGACCTTCCT
GGTGTCTTCTATTAATGGACGACTGGATACATTGCGAACGAGTCTCCTTTACGAAGTGTG
TAGGGAGTCACGACCAGAATTACGTCCGGGGGTAGGACCTGAAGGCACGCAGATGATACG
AAGCTAAGAAACGCCGCCACTACGGTACGCATTGTGGATGTAACGCCTCCAATCCTTGTG
TAGTGTGGCACGCGATGACGATATTCCTAATCGGCTTTGTAGTAGTTTATGCTGGTATT
```

```
CGGTGAGTCGGGTCTGAAGGCCATCAGTACCTATGTTAATGGAAGCCGCCGCTACTTGTG
GCTTCAGTACATACGGGACGATTATCCTAGCAGAAGCAAATCCTTAAATACGCGCGGCCG
ACCTTGCTGTTTGAGGGGAGTCCGATATTCAGCGTCTCATGGACTCCGTTGTACATTTGG
CCATAGTTTTTCGTGGGTAGGAGATAGATGTTAGGCGGAGATTATGTTTTATTTAATCCTT
GAGTCAGTGTGGCCGGGCTTACGGTGGTTAGTAGCTGCCGGTGAGAATGTCTACATTTTT
GATTCCCCGCCTGGCTACATTTGTTCTGAAGTCCAGTAACTAAGACCCTCAGCGTAGCAT
GGATCAATCGTGTAAAGGGCCGAAGCGATCCCTGTATAACTCTGTGAAGTAGTACATACAT
TCCTTAAATAATAACAGACTCCGGTGCT
```

Question 5e. What is the secret message?

Congratulations to the 2026 JHU Applied Genomics class!!!! Keep on looking for little green aliens...

Commands ran:

```
$ samtools faidx /Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/Assignment2/asm/asm/contigs.fasta 'NODE_3_length_41445_cov_18.207587:13851-14658' > insertion.fa
```

```
$ python3 dna-decode.py -d --input asm/insertion.fa
```

```
$ python3 dna-decode.py -d --input asm/insertion.fa --rev_comp
```